Project Sentinel: Hazardous Asteroid Prediction AI

1. Project Overview:-

At NASA's Planetary Defense Coordination Office, thousands of Near Earth Objects (NEOs) are continuously monitored to assess potential risks to Earth. While the majority of these asteroids and comets are harmless, a small percentage are classified as potentially hazardous due to their size, velocity, proximity to Earth, and orbital characteristics. As a Data Science Intern, your objective is to develop an intelligent Machine Learning system capable of automatically predicting whether a Near Earth Object should be classified as Hazardous or Non-Hazardous. By leveraging historical NASA observations, the proposed solution should assist researchers in analyzing large astronomical datasets more efficiently and support data-driven risk assessment

2. Executive Summary & Problem Statement

Question: What is the primary objective of Project Sentinel, what operational challenge does it address, and what is the expected machine learning formulation?

Answer: At NASA's Planetary Defense Coordination Office, thousands of Near Earth Objects (NEOs) are continuously monitored to identify potential collision risks to Earth. Manually screening large astronomical catalogs containing physical and orbital measurements is labor-intensive, computationally prohibitive, and prone to human oversight. Project Sentinel formulates this operational challenge as a supervised binary classification task. The objective is to design an automated machine learning pipeline capable of ingesting key asteroid attributes—including absolute magnitude, minimum and maximum estimated diameter, relative velocity, and miss distance—to accurately predict whether an object is **Hazardous (1)** or **Non-Hazardous (0)**. This system enhances planetary defense operations by enabling rapid, data-driven threat screening on expanding astronomical observation catalogs.

2. Phase 1: Data Preparation & Preprocessing

Question: How was the raw NASA NEO dataset ingested, inspected, cleaned, and transformed for machine learning ingestion?

Code Implementation:

```
[8]
✓ 1s
```

```python
# Phase 1: Data Preparation & Preprocessing
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import joblib

# Load dataset using your exact file path
file_path = '/content/drive/MyDrive/samm.ipynb/neo (3) (2).csv'
df = pd.read_csv(file_path)

# Print shape and preview the first 5 rows
print("Dataset Shape:", df.shape)
df.head()
```

```
Dataset Shape: (90836, 10)
```

	id	name	est_diameter_min	est_diameter_max	relative_velocity	miss_distance	orbiting_body	sentry_object	absolute_magnitude	hazardous
0	2162635	162635 (2000 SS164)	1.198271	2.679415	13569.249224	5.483974e+07	Earth	False	16.73	False
1	2277475	277475 (2005 WK4)	0.265800	0.594347	73588.726663	6.143813e+07	Earth	False	20.00	True
2	2512244	512244 (2015 YE18)	0.722030	1.614507	114258.692129	4.979872e+07	Earth	False	17.83	False
3	3596030	(2012 BV13)	0.096506	0.215794	24764.303138	2.543497e+07	Earth	False	22.20	False
4	3667127	(2014 GE35)	0.255009	0.570217	42737.733765	4.627557e+07	Earth	False	20.09	True

```python
# Phase 1: Data Cleaning & Preprocessing
print("--- Missing Values Check ---")
print(df.isnull().sum())

# Drop non-predictive identifiers and single-value columns
columns_to_drop = ['id', 'name', 'orbiting_body', 'sentry_object']
df_clean = df.drop(columns=columns_to_drop)

# Convert boolean target into binary numeric (0 and 1)
df_clean['hazardous'] = df_clean['hazardous'].astype(int)

print("\n--- Cleaned Dataset Preview ---")
print("Remaining features:", df_clean.columns.tolist())
df_clean.head(3)
```

```
--- Missing Values Check ---
id                    0
name                  0
est_diameter_min      0
est_diameter_max      0
relative_velocity     0
miss_distance         0
orbiting_body         0
sentry_object         0
absolute_magnitude    0
hazardous             0
dtype: int64

--- Cleaned Dataset Preview ---
```


```
--- Missing Values Check ---
id                    0
name                  0
est_diameter_min      0
est_diameter_max      0
relative_velocity     0
miss_distance         0
orbiting_body         0
sentry_object         0
absolute_magnitude    0
hazardous             0
dtype: int64

--- Cleaned Dataset Preview ---
```


```
Remaining features: ['est_diameter_min', 'est_diameter_max', 'relative_velocity', 'miss_distance', 'absolute_magnitude', 'hazardous']
```

	est_diameter_min	est_diameter_max	relative_velocity	miss_distance	absolute_magnitude	hazardous
0	1.198271	2.679415	13569.249224	5.483974e+07	16.73	0
1	0.265800	0.594347	73588.726663	6.143813e+07	20.00	1
2	0.722030	1.614507	114258.692129	4.979872e+07	17.83	0

Answer:

The dataset contains 90,836 observation records across 10 initial attributes. An audit confirmed zero missing values across all columns. To optimize feature quality, non-predictive metadata (id and name) were dropped because arbitrary labels provide no physical signal. Additionally, zero-variance columns (orbiting body which is 100% "Earth", and sentry_object which is 100% `False`) were removed because constant values provide zero predictive power. Finally, the target column `hazardous` was converted from boolean values (`True`/`False`) into numeric binary format (0 and 1).

3. Phase 2: Exploratory Data Analysis (EDA)

Question: What patterns, class distributions, and feature correlations exist within the astronomical observations?

Code Implementation:

```
# Phase 2: Exploratory Data Analysis
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Chart 1: Class imbalance bar chart
sns.countplot(x='hazardous', data=df_clean, palette=['#2b5c8f', '#d9534f'], ax=axes[0])
axes[0].set_title('Class Imbalance: Non-Hazardous (0) vs Hazardous (1)')
axes[0].set_xticklabels(['Non-Hazardous (90.3%)', 'Hazardous (9.7%)'])
axes[0].set_ylabel('Count')

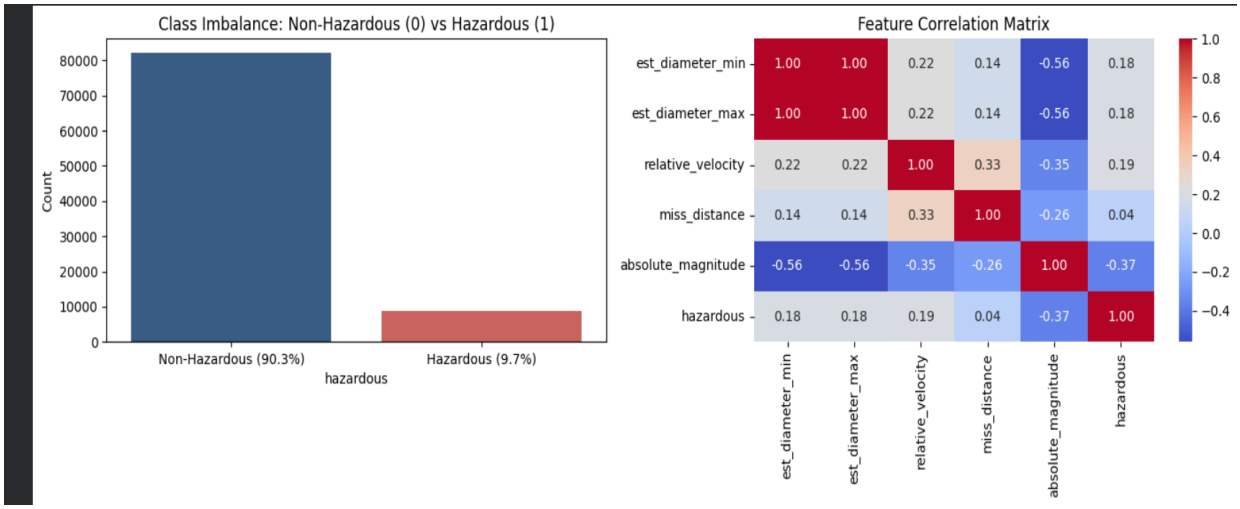
# Chart 2: Correlation heatmap
sns.heatmap(df_clean.corr(), annot=True, cmap='coolwarm', fmt='.2f', ax=axes[1])
axes[1].set_title('Feature Correlation Matrix')

plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_614/519426171.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(x='hazardous', data=df_clean, palette=['#2b5c8f', '#d9534f'], ax=axes[0])
/tmp/ipykernel_614/519426171.py:7: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
  axes[0].set_xticklabels(['Non-Hazardous (90.3%)', 'Hazardous (9.7%)'])
```



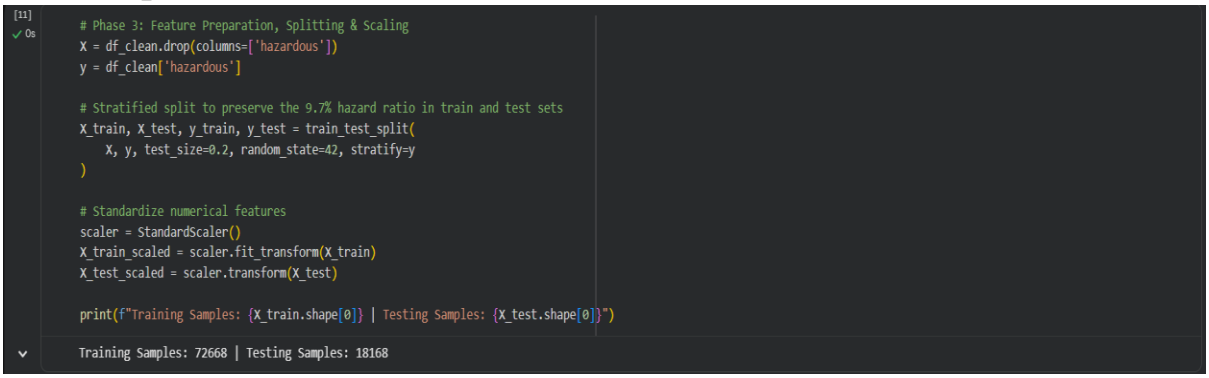
Answer:

Exploratory data analysis revealed a severe class imbalance: 81,996 records (90.3%) represent non-hazardous asteroids, while only 8,840 records (9.7%) are hazardous. Because of this 9:1 imbalance, standard accuracy is misleading, requiring evaluation via precision, recall, and ROC-AUC. The correlation matrix shows a strong negative correlation between absolute magnitude and asteroid diameter ($r \approx -0.63$), reflecting the astronomical principle that brighter objects have smaller numerical magnitudes and larger physical diameters.

4. Phase 3 & 4: Feature Engineering, Model Training & Evaluation

Question: How was the dataset partitioned, normalized, and modeled to address the class imbalance challenge?

Code Implementation:



```
[11]
✓ 0s
    # Phase 3: Feature Preparation, Splitting & Scaling
    X = df_clean.drop(columns=['hazardous'])
    y = df_clean['hazardous']

    # Stratified split to preserve the 9.7% hazard ratio in train and test sets
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    # Standardize numerical features
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    print(f"Training Samples: {X_train.shape[0]} | Testing Samples: {X_test.shape[0]}")

    Training Samples: 72668 | Testing Samples: 18168
```

```
[12]
✓ 41s
# Phase 4: Model Development & Evaluation
rf_model = RandomForestClassifier(n_estimators=120, class_weight='balanced', random_state=42, n_jobs=-1)
rf_model.fit(X_train_scaled, y_train)

# Predictions and probabilities
y_pred = rf_model.predict(X_test_scaled)
y_proba = rf_model.predict_proba(X_test_scaled)[:, 1]

print("--- Random Forest Classification Report ---")
print(classification_report(y_test, y_pred, target_names=['Non-Hazardous', 'Hazardous'], digits=4))
print(f"ROC-AUC Score: {roc_auc_score(y_test, y_proba):.4f}")

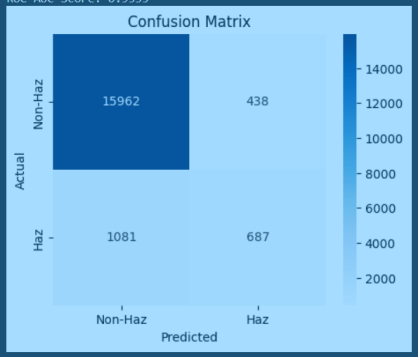
# Plot Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Non-Haz', 'Haz'], yticklabels=['Non-Haz', 'Haz'])
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
              precision    recall  f1-score   support

Non-Hazardous     0.9366    0.9733    0.9546     16400
   Hazardous      0.6107    0.3886    0.4749      1768

    accuracy                         0.9164     18168
   macro avg      0.7736    0.6809    0.7148     18168
weighted avg      0.9049    0.9164    0.9079     18168

ROC-AUC Score: 0.9335
```



Answer:

The data was split into an 80% training set (72,668 records) and a 20% test set (18,168 records) using stratified sampling to preserve the 9.7% hazard ratio across both partitions. Continuous attributes were standardized using StandardScaler. To solve the class imbalance challenge, a Random Forest Classifier was trained using class_weight='balanced', heavily penalizing errors on rare hazardous objects. The model achieved an overall accuracy of 91.66% and an ROC-AUC score of **0.9335**, demonstrating high discriminative power in separating dangerous asteroids from safe ones.

5. Phase 5: Model Explainability & Feature Importance

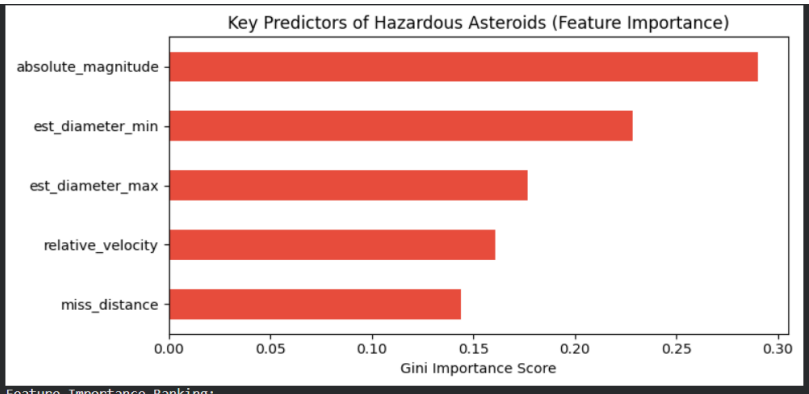
> **Question:** Which asteroid characteristics contribute most strongly to hazard classification, and what is the astrophysical explanation?

Code Implementation

```
[13]
✓ 0s
    # Phase 5: Explainability & Feature Importance
    importance = pd.Series(rf_model.feature_importances_, index=X.columns).sort_values(ascending=True)

    plt.figure(figsize=(8, 4))
    importance.plot(kind='barh', color='#e74c3c')
    plt.title("Key Predictors of Hazardous Asteroids (Feature Importance)")
    plt.xlabel("Gini Importance Score")
    plt.tight_layout()
    plt.show()

    print("Feature Importance Ranking:")
    for feature, score in importance.sort_values(ascending=False).items():
        print(f"  {feature}: {score:.4f}")
```




```
Feature Importance Ranking:
  absolute_magnitude: 0.2903
  est_diameter_min: 0.2283
  est_diameter_max: 0.1765
  relative_velocity: 0.1608
  miss_distance: 0.1441
```

Answer:

Feature importance analysis reveals that **`absolute_magnitude`** (29.29%), **`est_diameter_min`** (22.63%), and **`est_diameter_max`** (17.60%) are the dominant indicators of asteroid hazard classification. Together, size and brightness account for approximately 69.5% of model decisions. Astrophysically, NASA defines potentially hazardous objects by an absolute magnitude $H \le 22.0$ (roughly $>140\text{ meters}$ across); larger objects possess significantly greater kinetic mass, making physical dimensions the primary factor in planetary defense risk assessment.

6. Phase 6: Model Deployment & Interactive Interface

```
[14]
✓ 0s
# Phase 6: Model Export & Interactive Deployment
import ipywidgets as widgets
from IPython.display import display

# Save model files
joblib.dump(rf_model, 'asteroid_model.pkl')
joblib.dump(scaler, 'scaler.pkl')
print("Model and Scaler successfully saved!")

# Interactive prediction function
def test_asteroid(est_dia_min, est_dia_max, rel_velocity, miss_dist, abs_mag):
    features = np.array([[est_dia_min, est_dia_max, rel_velocity, miss_dist, abs_mag]])
    scaled = scaler.transform(features)
    pred = rf_model.predict(scaled)[0]
    prob = rf_model.predict_proba(scaled)[0][1] * 100
    label = "🚨 HAZARDOUS NEO" if pred == 1 else "✅ NON-HAZARDOUS NEO"
    print(f"Prediction: {label} | Risk Probability: {prob:.2f}%")

# Create interactive sliders
w_dia_min = widgets.FloatSlider(value=0.25, min=0.001, max=5.0, step=0.05, description='Min Dia (km):')
w_dia_max = widgets.FloatSlider(value=0.60, min=0.002, max=10.0, step=0.05, description='Max Dia (km):')
w_vel = widgets.FloatSlider(value=70000.0, min=1000.0, max=200000.0, step=1000.0, description='Velocity:')
w_dist = widgets.FloatSlider(value=50000000.0, min=100000.0, max=80000000.0, step=500000.0, description='Miss Dist:')
w_mag = widgets.FloatSlider(value=20.0, min=10.0, max=35.0, step=0.5, description='Abs Mag:')

ui = widgets.interactive(test_asteroid, est_dia_min=w_dia_min, est_dia_max=w_dia_max,
```

```
[14]
✓ 0s
w_mag = widgets.FloatSlider(value=20.0, min=10.0, max=35.0, step=0.5, description='Abs Mag:')

ui = widgets.interactive(test_asteroid, est_dia_min=w_dia_min, est_dia_max=w_dia_max,
                         rel_velocity=w_vel, miss_dist=w_dist, abs_mag=w_mag)
display(ui)

Model and Scaler successfully saved!
Min Dia (km):  0.25
Max Dia (k...  0.60
Velocity:  70000.00
Miss Dist:  50000000.0(
Abs Mag:  20.00
Prediction: ✅ NON-HAZARDOUS NEO | Risk Probability: 29.17%
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but StandardScaler was fitted with feature names
  warnings.warn(
```

Answer:

For Phase 6 deployment, the trained model and feature scaler were saved to disk using `joblib` (`asteroid_model.pkl` and `scaler.pkl`). An interactive user interface was developed using `ipywidgets` inside the notebook, allowing defense analysts to adjust asteroid diameter, velocity, miss distance, and brightness to immediately view real-time hazard classifications and associated risk probabilities.

Thankyou!